Graceful shutdown & monitoring

Why it matters

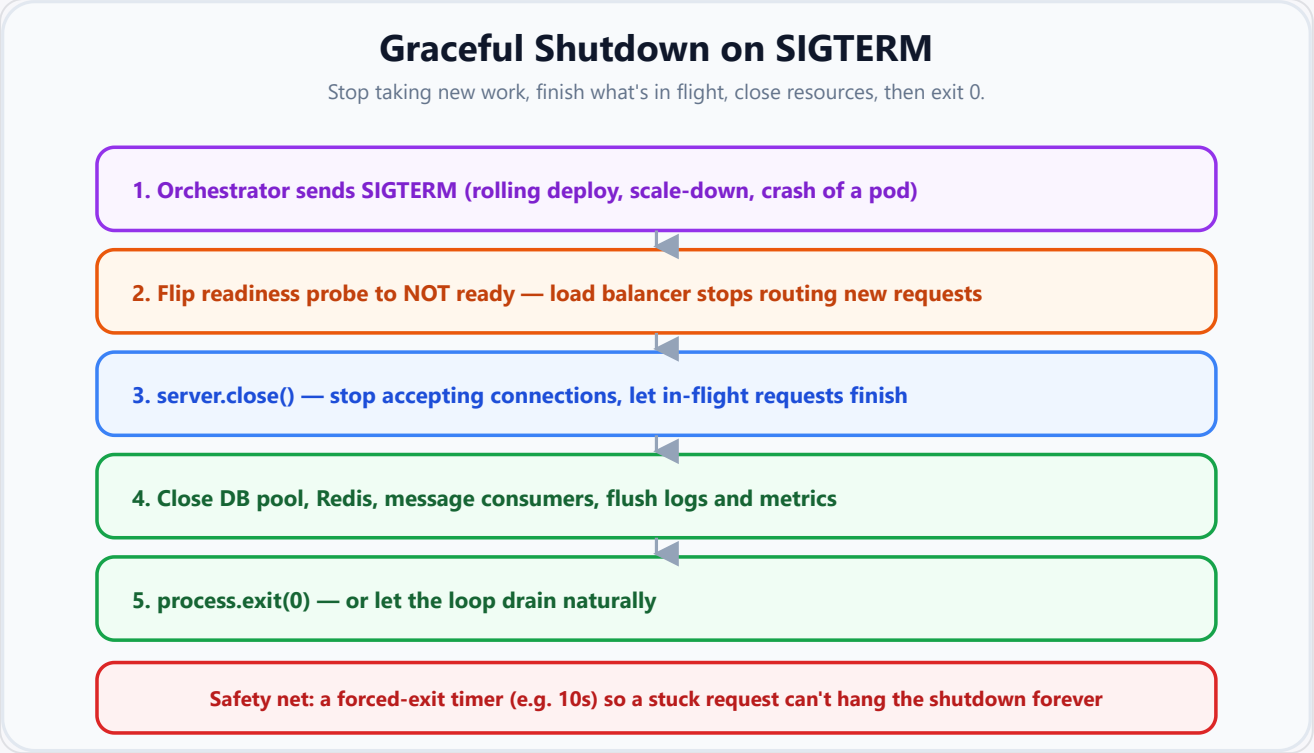
In a modern deployment, your process gets **killed constantly** — and that's normal. Every rolling deploy stops the old version, every autoscaling scale-down removes instances, every Kubernetes node drain evicts pods, every crash-and-restart cycle ends a process. The question is not *whether* your app will be told to stop, but *what happens to the requests in flight* when it is. Done naively, the process dies mid-request: a user's payment is cut off half-committed, a database write is left dangling, a queued job is lost. Done right — **graceful shutdown** — the app stops accepting new work, finishes what it's already doing, closes its connections cleanly, and exits. The difference is invisible when it works and a 3 a.m. incident when it doesn't.

The flip side is **monitoring and health checks**: the platform needs to know, continuously, whether your instance is alive and whether it's ready to receive traffic — and you need to know your service's latency, error rate, and resource use before users tell you. Together, graceful shutdown and health/monitoring are what make a Node service *operable*. In interviews they're a favourite because they reveal whether you've actually run software in production: "what happens when Kubernetes sends SIGTERM?" is a question you can only answer well if you've had to.

The concept, simply

When an orchestrator wants your process to stop, it sends the **SIGTERM** signal — a polite "please shut down." (Contrast **SIGKILL**, which can't be caught and kills instantly, and **SIGINT**, what Ctrl-C sends in your terminal.) By default Node exits on SIGTERM immediately, abandoning in-flight work. Graceful shutdown means *catching* SIGTERM and running an orderly teardown before exiting.

logs/metrics, **(4)** exit with code 0. And critically, **(5)** a *forced-exit timer*: if draining takes too long (a stuck request), a timeout forces the exit so shutdown can't hang forever.



Graceful shutdown on SIGTERM: mark not-ready so the LB stops routing, stop accepting new connections, drain in-flight requests, close DB/Redis/consumers, then exit — with a forced-exit timer as a safety net.

Health checks come in two flavours that people constantly conflate. A **liveness** probe answers "is the process alive, or wedged and needing a restart?" — fail it and the platform kills+restarts the instance. A **readiness** probe answers "is this instance ready to receive traffic *right now*?" — fail it and the load balancer stops routing to it *without* killing it. The distinction is what makes zero-downtime shutdown work: on SIGTERM you flip *readiness* to not-ready first, so the load balancer drains you, while the process stays up long enough to finish current requests.

Monitoring is the always-on version of the same instinct: continuously observe the service through metrics (latency, throughput, error rate, event-loop lag, memory), logs (structured, to stdout), and traces, so you can see problems before users do and diagnose them fast.

How it works

You register handlers for the termination signals. On SIGTERM (and usually SIGINT for local dev), you kick off the shutdown sequence. The linchpin is `server.close()`: it tells the HTTP server to *stop accepting new connections* but keeps existing ones open until their in-flight requests complete, then fires its callback. That callback is where you close everything else — `await pool.end()`, `await redis.quit()`, stop queue consumers, flush your metrics/log buffers — and finally exit.

Two subtleties make this actually work in production. First, **keep-alive connections**: idle keep-alive sockets can keep `server.close()` from ever completing because the connection is open with no active request. Node's `server.closeIdleConnections()` (and, more aggressively, `closeAllConnections()`) handle this; libraries and Node's own `server.keepAliveTimeout` also help. Second, the **ordering with the load balancer**: if you call `server.close()` before the LB has noticed you're going away, new requests it already routed can hit a closing server. That's why you flip the readiness probe to not-ready *first*, wait a beat for the LB to stop sending traffic, and only then stop accepting. In Kubernetes this is also handled by the pod's `preStop` hook and `terminationGracePeriodSeconds`.

The **forced-exit timer** is non-negotiable: `setTimeout(() => process.exit(1), 10_000).unref()`. If a request hangs (a slow upstream, a stuck DB query), you don't want shutdown to block indefinitely — after the grace period, exit anyway. The `.unref()` ensures this timer itself doesn't keep the process alive if everything *else* finished cleanly.

Health endpoints are ordinary HTTP routes. A liveness route (`/healthz`) returns 200 as long as the process can respond — keep it cheap and dependency-free so a slow DB doesn't cause needless restarts. A readiness route (`/readyz`) checks that critical dependencies (DB, Redis) are reachable *and* returns 503 once shutdown has begun, so the LB drains you. For monitoring, you expose metrics (commonly a Prometheus `/metrics` endpoint via `prom-client`), and you watch **event-loop lag** specifically — rising lag is Node's canary for a blocked event loop, the failure this whole chapter's other topics exist to prevent.

Finally, handle the **crash signals**: `uncaughtException` and `unhandledRejection`. Best practice is to log, attempt a fast cleanup, and exit — because after an uncaught exception the process is in an unknown state, so you let the supervisor restart a fresh one rather than limp on corrupted.

Code examples

Node.js Backend Kit

A complete, production-shaped graceful shutdown: readiness flip, drain, resource cleanup, forced-exit safety net.

JAVASCRIPT

Copy

```
import http from 'node:http';

let shuttingDown = false;
const server = http.createServer(app);
server.keepAliveTimeout = 5000;                    // don't let idle sockets
server.listen(3000);

async function shutdown(signal) {
  if (shuttingDown) return;                        // idempotent — signals c
  shuttingDown = true;
  console.log(`${signal} received — shutting down`);

  // 1) Force-exit safety net so a stuck request can't hang us forever.
  const killer = setTimeout(() => {
    console.error('Drain timed out — forcing exit');
    process.exit(1);
  }, 10_000);
  killer.unref();

  // 2) Stop accepting new connections; finish in-flight requests.
  server.close(async () => {
    try {
      await pool.end();                            // close DB pool
      await redis.quit();                          // close Redis
      console.log('Clean shutdown complete');
      process.exit(0);
    } catch (err) {
      console.error('Error during cleanup', err);
      process.exit(1);
    }
  });
  server.closeIdleConnections?.();                 // release idle keep-aliv
}
```

Liveness vs readiness endpoints — note how readiness reports not-ready as soon as shutdown starts, so the load balancer drains this instance.

JAVASCRIPT Copy

```
// Liveness: is the process responsive at all? Keep it cheap, no deps.
app.get('/healthz', (req, res) => res.status(200).send('ok'));

// Readiness: should traffic be routed here right now?
app.get('/readyz', async (req, res) => {
  if (shuttingDown) return res.status(503).send('shutting down');  // drai
  try {
    await pool.query('SELECT 1');                // critical dep reachable
    await redis.ping();
    res.status(200).send('ready');
  } catch {
    res.status(503).send('dependencies unavailable');
  }
});
```

Minimal monitoring: expose Prometheus metrics and watch event-loop lag, Node's early warning for a blocked loop.

JAVASCRIPT Copy

```
import client from 'prom-client';
import { monitorEventLoopDelay } from 'node:perf_hooks';

client.collectDefaultMetrics();                  // CPU, memory, GC, handl
const loopLag = monitorEventLoopDelay({ resolution: 20 });
loopLag.enable();

const lagGauge = new client.Gauge({
  name: 'nodejs_eventloop_lag_p99_ms',
  collect() { this.set(loopLag.percentile(99) / 1e6); },  // ns → ms
});
```

```
});
```

Crash handlers — log and exit so the supervisor restarts a clean process rather than continuing in an unknown state.

JAVASCRIPT Copy

```
process.on('uncaughtException', (err) => {
  console.error('uncaughtException — exiting', err);
  process.exit(1);                                  // let PM2/k8s restart a
});
process.on('unhandledRejection', (reason) => {
  console.error('unhandledRejection — exiting', reason);
  process.exit(1);
});
```

Common mistakes & gotchas

- **Not handling SIGTERM at all.** Node's default is immediate exit, so every deploy/scale-down kills in-flight requests. You must catch it and drain.
- **No forced-exit timer.** If a request hangs, `server.close()` never calls back and the process hangs until the platform SIGKILLs it — losing the graceful part entirely. Always add an unref'd timeout.
- **Idle keep-alive sockets block `server.close()`.** Without closing idle connections (or a keep-alive timeout), drain can stall on connections with no active request.
- **Confusing liveness and readiness.** Putting dependency checks in the *liveness* probe means a brief DB blip triggers pointless restarts. Deep checks belong in readiness; liveness should just prove the process responds.
- **Closing the server before the LB stops routing.** Flip readiness to not-ready first and wait a moment, or new requests hit a closing server. (In k8s, also mind `preStop` and grace period.)

- **Trying to continue after `uncaughtException`.** The process state is unknown; limping on risks corrupt data. Log and exit; let the supervisor restart.
- **Not making the handler idempotent.** Signals can arrive more than once; without a guard you start shutdown twice.
- **Ignoring event-loop lag.** High lag means the loop is blocked (CPU-bound code) — the one metric most specific to Node health, and the one people forget to watch.
- **Grace period mismatch.** If your drain timeout exceeds the platform's `terminationGracePeriodSeconds`, the platform SIGKILLs you mid-drain. Keep your forced-exit shorter than the platform grace period.

Best practices

- **Catch SIGTERM (and SIGINT) and run an ordered drain**: readiness→not-ready, `server.close()`, close DB/Redis/consumers, exit 0.
- **Always add an unref'd forced-exit timer** shorter than the platform's grace period.
- **Close idle keep-alive connections** so drain can complete; set a sane `keepAliveTimeout`.
- **Separate liveness (cheap, no deps) from readiness (checks deps + reports draining).**
- **Make the shutdown handler idempotent** and log each step so you can debug slow drains.
- **On `uncaughtException` / `unhandledRejection`: log and exit**; rely on the supervisor to restart clean.
- **Expose metrics (Prometheus/OpenTelemetry) and watch event-loop lag, error rate, latency, memory.** Alert on symptoms users feel.
- **Log structured JSON to stdout** (12-factor) and let the platform aggregate; include request IDs for tracing.
- **Test shutdown** — send a real SIGTERM under load and confirm in-flight requests complete and connections close.

⭐ Interview lens

Q: What happens when Kubernetes sends your pod SIGTERM, and what should happen?

A: By default Node exits immediately, killing in-flight requests. It *should* catch SIGTERM and drain: flip readiness to not-ready so the LB stops routing, call `server.close()` to stop accepting new connections while finishing current ones, close DB/Redis/consumers, then exit 0 — with a forced-exit timer (shorter than `terminationGracePeriodSeconds`) as a safety net.

Q: Liveness vs readiness probe — what's the difference?

A: Liveness answers "is the process alive or wedged?" — failing it restarts the instance. Readiness answers "should it get traffic right now?" — failing it removes it from the load balancer *without* restarting. You flip readiness to not-ready on shutdown to drain traffic while the process finishes current work. (Common trap: putting DB checks in liveness, causing restart storms on a DB blip.)

Q: Why do you need a forced-exit timeout during shutdown?

A: Because `server.close()` only fires once all in-flight requests finish; a single stuck request (slow upstream, hung query) would block shutdown forever, and the platform would eventually SIGKILL you. An unref'd `setTimeout(process.exit, N)` guarantees you exit within a bounded time.

Q: Should you keep running after an uncaughtException?

A: No. After an uncaught exception the process is in an undefined state — continuing risks corrupt data and hidden bugs. Log it, attempt fast cleanup, and exit so the supervisor (PM2/k8s) starts a fresh process. Use uncaughtException as a last-resort log-and-die, not as normal error handling.

Q: Why does `server.close()` sometimes never complete?

A: It waits for all open connections to go idle, but idle HTTP keep-alive sockets stay open with no active request, so they never "finish." Use

server.closeIdleConnections() (and a keepAliveTimeout) so drain can complete. This surprises a lot of people.

Q: What Node-specific metric would you monitor and why?

A: Event-loop lag/delay. Because Node runs your code on one thread, rising loop lag means the loop is blocked by synchronous/CPU-bound work — requests are queuing even if CPU looks fine. It's the earliest, most Node-specific signal of the exact failure that worker_threads/clustering exist to avoid. (Alongside standard latency, error rate, and memory.)

Cheat-card

- **SIGTERM** = polite stop (catch it); **SIGKILL** = uncatchable; **SIGINT** = Ctrl-C.
- Drain order: readiness→not-ready → `server.close()` → close DB/Redis/consumers → `exit(0)`.
- **Always** add an unref'd **forced-exit timer**, shorter than the platform grace period.
- `server.close()` can hang on idle keep-alives → `closeIdleConnections()` + `keepAliveTimeout`.
- **Liveness** = alive? (cheap, restarts) vs **Readiness** = take traffic? (deps + reports draining, no restart).
- `uncaughtException` / `unhandledRejection` → log and **exit**; let the supervisor restart.
- Monitor **event-loop lag** (Node's canary) + latency, error rate, memory; logs as JSON to stdout.
- Make the handler **idempotent**; test shutdown under real load.

QUICK CHECK — no marks, just for you.

1. What is the difference between a liveness probe and a readiness probe?

- ◯ Liveness checks dependencies; readiness checks whether the process is alive

- They are the same check exposed on two different routes
- Liveness removes traffic; readiness restarts the pod

Check answer

2. Why must a graceful shutdown always include an unref'd forced-exit timer?

- To restart the process automatically after shutdown
- To keep the process alive indefinitely so no request is ever dropped
- Because a single stuck request would make server.close() never call back, so a timeout forces the exit within a bounded time before the platform SIGKILLs you
- To flush logs before the readiness probe flips

Check answer

3. Which Node-specific metric is the earliest signal of a blocked event loop?

- Event-loop lag/delay — rising lag means synchronous/CPU-bound work is blocking the single-threaded loop even if CPU looks fine
- Total heap size, which spikes whenever the loop blocks
- Open file descriptor count
- HTTP 500 error rate, which always precedes loop blocking

Check answer

Previous
PM2 & Docker

Next
Real-time with WebSockets

LEARN MORE

ARTICLES & DOCS

- Node.js process: signal events (SIGTERM/SIGINT) & exit
  nodejs.org
- Node.js http: server.close() for draining connections
  nodejs.org

VIDEOS



Here's how to Gracefully Shutdown your apps (with Node.js examples)
Software Developer Diaries

Hand-picked references for this lesson.